

Server Monitoring tool

1. *Install and check the functioning of server monitoring tools like:*
 1. Nagios
 2. Zabbix
 2. *Deeply study the commands used to monitor the server as well as the services.*
-

Part 1: Zabbix Monitoring Setup

This section provides instructions for deploying a multi-node Zabbix deployment architecture on AWS EC2 nodes utilizing Red Hat Enterprise Linux (RHEL)-based systems.

1.1 Target Architecture Topology

- **Server EC2 Instance:** Hosts the core Zabbix Server software components, Apache Web Server Engine, PHP-FPM execution wrapper, and MariaDB relational storage backend.
- **Client/Target EC2 Instance:** Hosts the native Zabbix Agent software layer daemon optimized for resource metrics parsing.

1.2 Server Architecture Installation Steps

Step 1: Bootstrap the official system release packages for Zabbix 7.0 LTS repository setup:

```
rpm -Uvh https://repo.zabbix.com/zabbix/7.0/rhel/9/x86_64/zabbix-release-latest-7.0.el9.noarch.rpm
dnf clean all
```

Step 2: Deploy foundational stack packages including Zabbix binaries, Apache, MariaDB Server instance, and PHP modules:

```
dnf install zabbix-server-mysql zabbix-web-mysql zabbix-apache-conf zabbix-sql-scripts zabbix-agent mariadb-server httpd php php-mysqlnd php-gd php-xml php-bcmath php-mbstring -y
```

Step 3: Toggle boot-persistent startup flags and activate the MariaDB runtime instance daemon:

```
systemctl enable --now mariadb
```

Step 4: Provision secure schema isolation and database credentials inside MariaDB:

```
mysql -u root -p

create database zabbix character set utf8mb4 collate utf8mb4_bin;
create user zabbix@localhost identified by 'Password@123';
grant all privileges on zabbix.* to zabbix@localhost;
flush privileges;
exit;
```

Step 5: Decompress and import structural boilerplate schemas and initial datasets directly into the newly created database storage system:

```
zcat /usr/share/zabbix-sql-scripts/mysql/server.sql.gz | mysql -uzabbix -p zabbix
```

Step 6: Inject the custom database authentication credentials into the system daemon configuration mapping block via modifications to zabbix_server.conf:

```
vi /etc/zabbix/zabbix_server.conf

# Modify the parameter inside the file to look as follows:
DBPassword=Password@123
```

Step 7: Bootstrap and start the complete operational infrastructure suite across all monitoring tiers concurrently:

```
systemctl enable --now zabbix-server zabbix-agent httpd php-fpm
```

1.3 Web UI Wizard Walkthrough Configuration

Launch your preferred browser workspace interface navigation pointer target and specify your instance host node path:

```
http://SERVER-PUBLIC-IP/zabbix/
```

- **Database Setup Parameters:** Database Host: localhost | Database Name: zabbix | User: zabbix | Password: Password@123

- **Default Access Framework Credentials:** Username: Admin | Password: zabbix

1.4 Client Node Agent Provisioning

Step 1: Execute repo definition installations matching server setup on your designated client system and pull zabbix-agent binaries:

```
rpm -Uvh https://repo.zabbix.com/zabbix/7.0/rhel/9/x86_64/zabbix-release-latest-7.0.el9.noarch.rpm
dnf clean all
dnf install zabbix-agent -y
```

Step 2: Establish connection routing configurations inside the internal application runtime configuration environment setup profile:

```
vi /etc/zabbix/zabbix_agentd.conf

# Configure the directive definitions inside the file text:
Server=SERVER_PRIVATE_IP
ServerActive=SERVER_PRIVATE_IP
Hostname=zabbix-client
```

Step 3: Enable the boot persistent flag state and start execution loops for the target system client monitoring module:

```
systemctl enable --now zabbix-agent
```

1.5 AWS Perimeter Rules & UI Registrations

- **Security Group Rules:** Expose inbound rule Port 10050 TCP traffic matching the Source context pointing directly to your Zabbix Server private IP interface address boundaries explicitly.
- **Dashboard Registration Strategy:** Navigate directly through the navigation structure options: Data collection → Hosts → Create host.
- **Target Specifications:** Define matching system Hostname string metrics, map standard baseline 'Linux by Zabbix agent template' structures, and reference target Client private IP metrics.

1.6 Hardware Infrastructure Stress-Test Validation

To confirm functional execution profiles, run structural test cycles to evaluate metric reporting accuracy on the target:

- **CPU Infrastructure Stress Loop:** Execute intensive looping sequences across computational threads:

```
yes > /dev/null & yes > /dev/null & yes > /dev/null & yes > /dev/null &  
# To terminate tests:  
pkill yes
```

- **Volatile Memory Stress Injection:** Allocate block space structures directly inside shared memory layout contexts:

```
dd if=/dev/zero of=/dev/shm/test bs=1M count=400  
# To clean up allocated space memory blocks:  
rm -f /dev/shm/test
```

- **Storage I/O Subsystem Strain:** Drive intensive disk read/write loops down inside core directories:

```
dd if=/dev/zero of=/tmp/testfile bs=1M count=5000  
# To remove test block allocations:  
rm -f /tmp/testfile
```

Verification: Access Web UI Dashboard paths via Monitoring → Hosts → Graphs to confirm metrics collection trends (CPU Spike, Memory Variations, Storage activity graphs).

Part 2: Nagios Monitoring Setup

2.1 Target Architecture Infrastructure Mapping

- **Server Infrastructure Core Node:** Hosts Core Nagios system orchestrator engines, Apache server utilities, and standard system compile dependencies.
- **Client Target Instance Stack:** Hosts local structural Nagios execution plugin metrics alongside specific remote execution monitoring packages (NRPE).

2.2 Source Build, Compile, and Server System Installation

Step 1: Set up underlying compilation frameworks, graphical engines, and structural utility packages on your host system node:

```
dnf install httpd php gcc glibc glibc-common wget unzip make gd gd-devel openssl-devel perl -y
```

Step 2: Generate target operational application user definitions alongside command privilege system groups mapping rules context structures:

```
useradd nagios
groupadd nagcmd
usermod -a -G nagcmd nagios
usermod -a -G nagcmd apache
```

Step 3: Retrieve official target build tarballs for Nagios Core v4.5.1 and unpack system archive files within internal build areas:

```
cd /tmp
wget https://assets.nagios.com/downloads/nagioscore/releases/nagios-4.5.1.tar.gz
tar -xzf nagios-4.5.1.tar.gz
cd nagios-4.5.1
```

Step 4: Execute source compilation flags mapping arguments configuration parameters rules and apply infrastructure generation binaries routines:

```
./configure --with-command-group=nagcmd
make all
make install
make install-init
make install-config
make install-commandmode
```

```
make install-webconf
```

Step 5: Setup explicit password definitions mapping authorization logic contexts for the web-accessible UI operator user profile:

```
htpasswd -c /usr/local/nagios/etc/htpasswd.users nagiosadmin
```

Step 6: Download, build, and deploy the core functional plugins codebase pack needed for base execution monitoring scripts:

```
cd /tmp
wget https://nagios-plugins.org/download/nagios-plugins-2.4.6.tar.gz
tar -xzf nagios-plugins-2.4.6.tar.gz
cd nagios-plugins-2.4.6
./configure --with-nagios-user=nagios --with-nagios-group=nagios
make
make install
```

Step 7: Launch web framework engines and initiate execution sequences for the core Nagios orchestrator process loop:

```
systemctl restart httpd
systemctl enable nagios
systemctl start nagios
```

Verification: Confirm monitoring interface interactions by executing browser navigation targeting URL: <http://SERVER-PUBLIC-IP/nagios>

2.3 Remote Client Engine Build and Setup (NRPE)

Step 1: Download, configure, and install the native remote processing application daemon suite (NRPE) directly on target client systems:

```
cd /tmp
wget https://github.com/NagiosEnterprises/nrpe/releases/download/nrpe-4.1.0/nrpe-4.1.0.tar.gz
tar -xzf nrpe-4.1.0.tar.gz
cd nrpe-4.1.0
./configure --disable-ssl --enable-command-args
make all
make install-groups-users
```

```
make install
make install-config
make install-init
```

Step 2: Append target orchestration host network identification markers to the trusted execution parameters file mapping layout framework:

```
vi /usr/local/nagios/etc/nrpe.cfg

# Bind the trusted operational boundaries mapping values block context:
allowed_hosts=127.0.0.1,::1,NAGIOS_SERVER_PRIVATE_IP
```

Step 3: Toggle persistent service boot startup definitions and run execution parameters loops for the remote metric broker system:

```
systemctl enable nrpe
systemctl start nrpe
```

- **Perimeter Firewall Rules:** Configure AWS Security Group policies to allow inbound TCP traffic targeting port 5666 from your Nagios Server IP address.

2.4 Server End Integration Configuration

Step 1: Compile check_nrpe test modules execution binaries directly on the centralized orchestration server endpoint nodes:

```
cd /tmp
wget https://github.com/NagiosEnterprises/nrpe/releases/download/nrpe-4.1.0/nrpe-4.1.0.tar.gz
tar -xzf nrpe-4.1.0.tar.gz
cd nrpe-4.1.0
./configure --disable-ssl
make check_nrpe
make install-plugin
```

Step 2: Verify active network command loop parsing functionalities targeting your managed endpoint nodes via check_nrpe test execution:

```
/usr/local/nagios/libexec/check_nrpe -n -H CLIENT_PRIVATE_IP
```

Step 3: Incorporate specific monitoring properties within a dedicated client configuration mapping document framework layout file:

```
vi /usr/local/nagios/etc/objects/client.cfg

# Append explicit object tracking structure mappings rules configuration layout
block:
define host {
    use                linux-server
    host_name          nagios-client
    alias              Nagios Client
    address            CLIENT_PRIVATE_IP
    check_command       check_nrpe! -n
}
```

- **Monitored Metric Targets:** CPU Load, Disk Usage percentage parameters, User session contexts, Active process lists, Zombie processes, SSH connection check loops, and ICMP Ping connectivity checks.

Step 4: Integrate global operational definition values inside system commands framework mapping lists structure locations:

```
vi /usr/local/nagios/etc/objects/commands.cfg

# Configure custom execution commands wrappers profiles:
define command {
    command_name      check_nrpe
    command_line       $USER1$/check_nrpe -H $HOSTADDRESS$ $ARG1$
}
define command {
    command_name      check_nrpe_arg
    command_line       $USER1$/check_nrpe -n -H $HOSTADDRESS$ -c $ARG1$
}
```

Step 5: Run data verification logic routines against global system mappings parameters settings profiles prior to execution loops activation:

```
/usr/local/nagios/bin/nagios -v /usr/local/nagios/etc/nagios.cfg
```

Step 6: Apply and load new production parameters by restarting your monitoring system orchestration process wrapper loop:


```
systemctl restart nagios
```

2.5 Service Polling Acceleration Tuning

To minimize delay when verifying configurations, adjust threshold boundaries inside the monitoring tracking definition layout block rules:

```
check_interval      0.25
retry_interval      0.25
max_check_attempts  2
```

Part 3: Deep Study of Server & Service Monitoring Commands

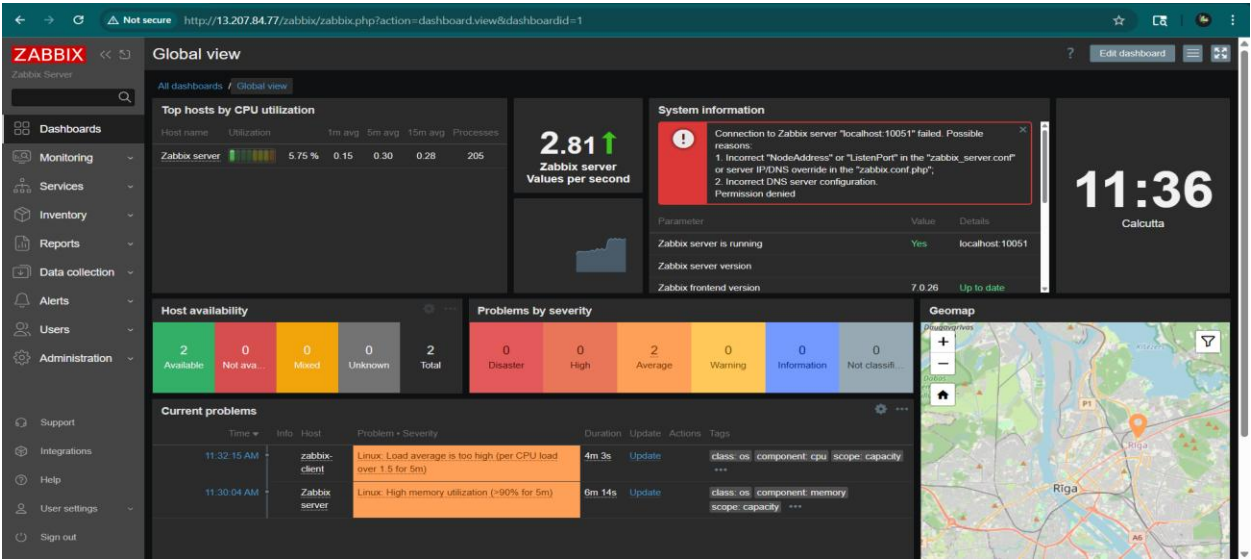
This section contains a deep dive into Linux system diagnostic utilities used to analyze server states, infrastructure pipelines, and application performance profiles.

Monitoring Domain	Core Linux Command Line Utilities	Operational Focus & Practical Application Metrics
CPU & Performance Monitoring	top htop mpstat sar uptime	• top: Displays dynamic real-time view of running system processes, CPU utilization fractions, and process priority maps. • htop: Interactive, text-based process viewer offering visual color-coded representation of multiple core utilization tracking. • mpstat: Dissects performance reporting breakdowns for individual or clustered processors. • sar: System Activity Reporter tool used to gather, log, and playback long-term operational framework usage data metrics. • uptime: Provides summary view of host execution duration metrics, user counts, and average resource queue length values over 1, 5, and 15 minutes.
Memory Allocation Diagnostics	free -h vmstat	• free -h: Provides parsed breakdowns of active, idle, cached, and swap memory configurations using human-readable metrics formatting scaling metrics. • vmstat: Outputs high-frequency statistics logs tracking processes, virtual swap execution tables, disk caching performance, and processor context switches.
Disk Subsystem & Storage I/O	df -h du -sh * iostat lsblk	• df -h: Identifies physical and virtual block mount definitions along with total consumption metrics profiles. • du -sh *: Evaluates isolated specific directory trees to locate high-density file paths. • iostat: Monitors system input/output device loading patterns to identify physical block tracking latency bottlenecks. • lsblk: Displays structured structural hierarchical overview maps matching mapped layout geometries of storage hardware block assets.
Network Interfaces & Sockets	ss -tulnp netstat -tulnp ip a	• ss -tulnp: Scans underlying socket connections, interface ports, and execution IDs directly out from kernel spaces. • netstat -tulnp: Traditional protocol mapping

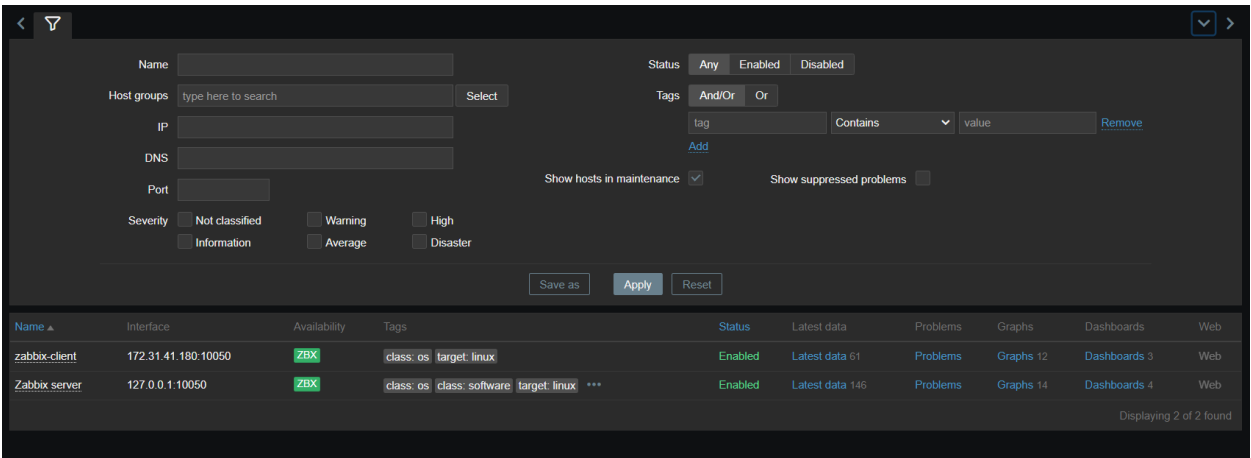
		analysis application suite for investigating interface connection listings and active listening configurations. • ip a: Examines IP bindings, virtual routing attachments, interface configurations, and layer state definitions.
Service & Runtime Management	<pre>systemctl status <service> journalctl -xe</pre>	• systemctl status: Displays active runtime state conditions, process control tracking, and system operational flags metadata contexts. • journalctl -xe: Pulls system log events with diagnostic error reports and execution tracking metadata annotations near the point of tail failure events.
Process Lifecycles & Controls	<pre>ps aux pstree kill / pkill</pre>	• ps aux: Captures static system snapshots containing all active application execution instances mapped to parent user environments. • pstree: Renders structural hierarchical process relationship views formatted as layout dependency tree branches. • kill / pkill: Dispatches execution signal flags to target process instances using either explicit PID numerical references or string pattern match parameters.
System Log Stream Tracking	<pre>tail -f /var/log/messages tail -f /var/log/secure</pre>	• /var/log/messages: Centralized global operating logs repository capturing core system events and generic application reports. • /var/log/secure: Dedicated security-auditing event stream capturing user connection handshakes, authentication evaluations, and privilege validation tracking loops.

Output Pictures

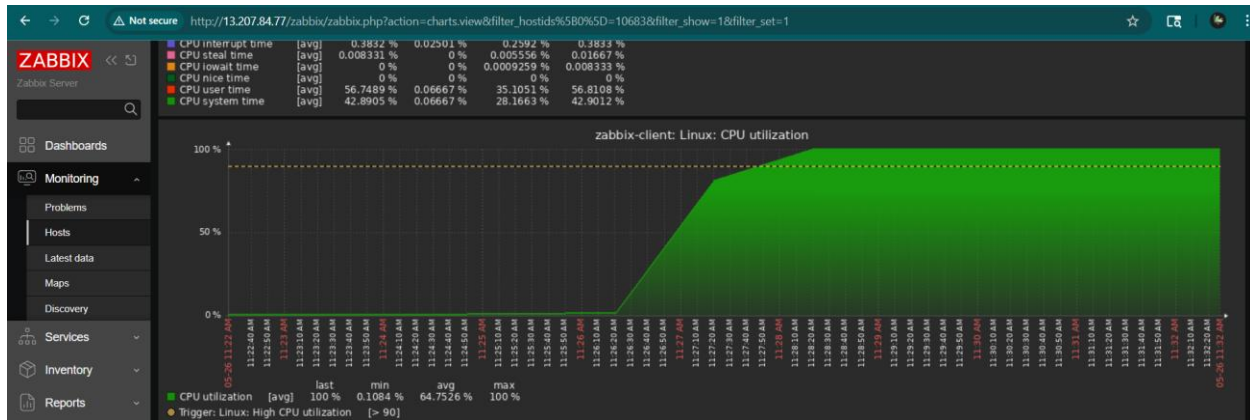
Zabbix dashboard



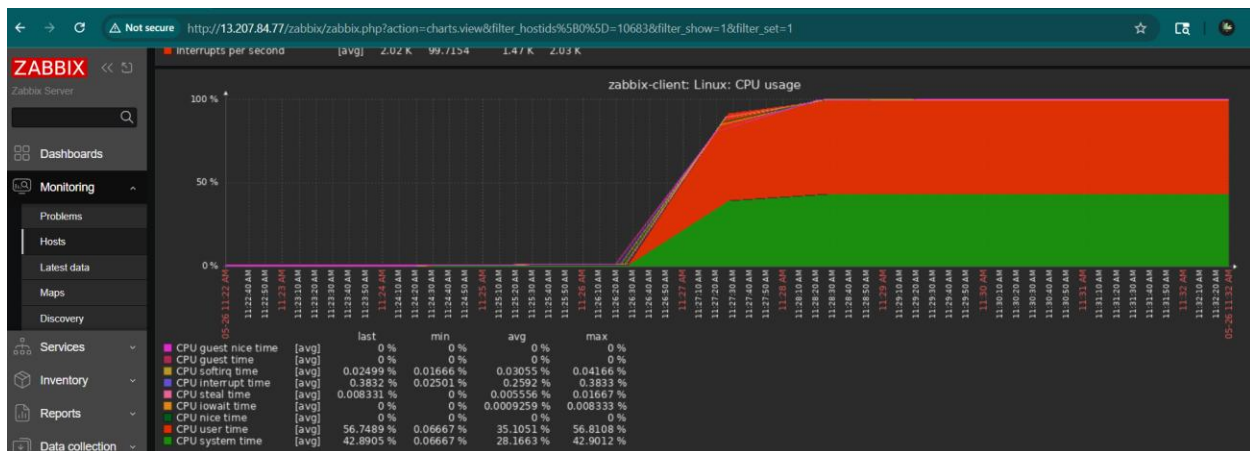
Zabbix client and server connected



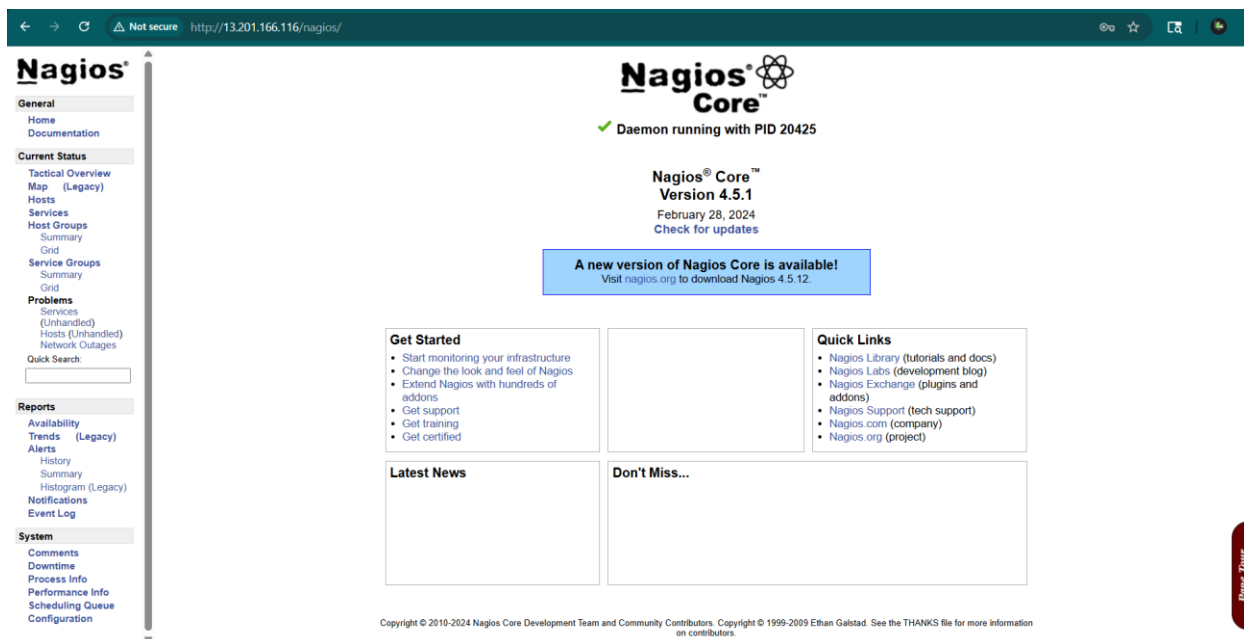
Zabbix-client CPU utilization



Zabbix-client cpu usage

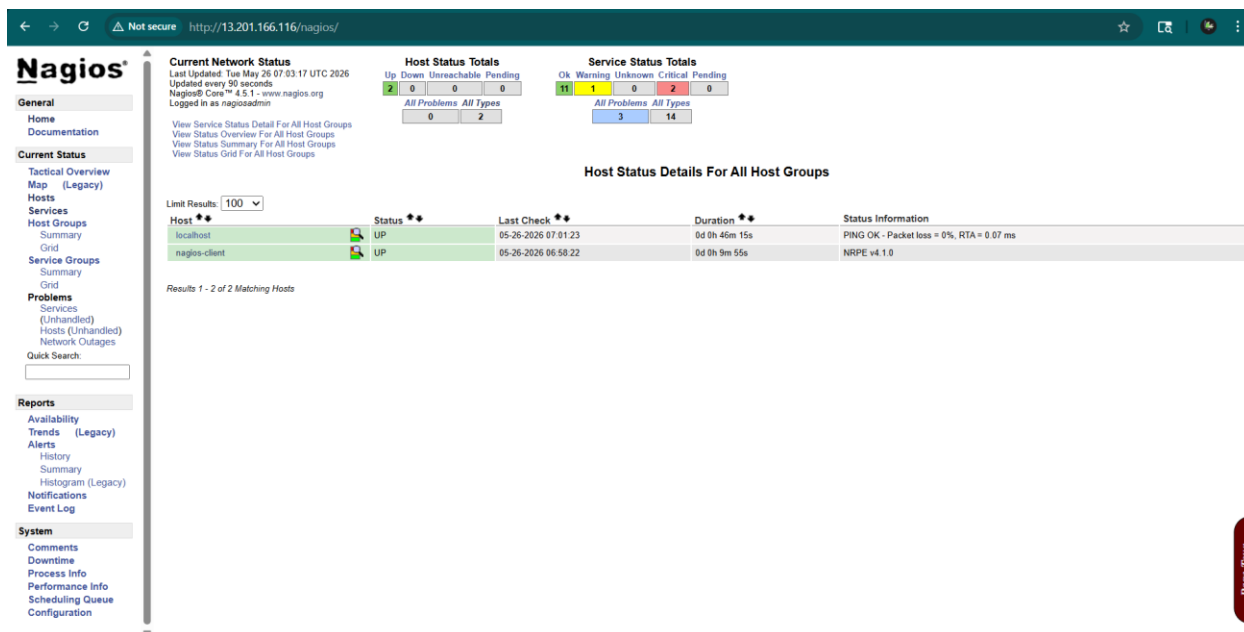


Nagios Dashboard



The screenshot shows the Nagios Core Version 4.5.1 dashboard. The top navigation bar includes a left sidebar with categories: General (Home, Documentation), Current Status (Tactical Overview, Map (Legacy), Hosts, Services, Host Groups, Summary, Grid, Service Groups, Summary, Grid), Problems (Services (Unhandled), Hosts (Unhandled), Network Outages), Quick Search, Reports (Availability, Trends (Legacy), Alerts, History, Summary, Histogram (Legacy), Notifications, Event Log), and System (Comments, Downtime, Process Info, Performance Info, Scheduling Queue, Configuration). The main content area features the Nagios Core logo, a status message 'Daemon running with PID 20425', the version 'Version 4.5.1' dated 'February 28, 2024', and a 'Check for updates' link. A blue banner announces 'A new version of Nagios Core is available! Visit nagios.org to download Nagios 4.5.12.' Below this are four sections: 'Get Started' with links to start monitoring, change look/feel, extend with addons, get support, training, and certification; 'Quick Links' to Nagios Library, Labs, Exchange, Support, company, and project; 'Latest News'; and 'Don't Miss...'. A 'Page Tour' button is on the right. The footer contains copyright information for 2010-2024.

Nagios client and server Up



This screenshot displays the 'Host Status Details For All Host Groups' page in Nagios Core. The left sidebar is identical to the dashboard view. The main content area includes 'Current Network Status' (Last Updated: Tue May 26 07:53:17 UTC 2026, Updated every 50 seconds, Nagios® Core™ 4.5.1 - www.nagios.org, Logged in as nagiosadmin), 'Host Status Totals' (Up: 2, Down: 0, Unreachable: 0, Pending: 0, All Problems: 2), and 'Service Status Totals' (OK: 15, Warning: 1, Unknown: 0, Critical: 2, Pending: 0, All Problems: 14). The 'Host Status Details' table shows two hosts: 'localhost' and 'nagios-client', both with a status of 'UP'. The table columns are Host, Status, Last Check, Duration, and Status Information. A 'Limit Results' dropdown is set to 100. The footer shows 'Results 1 - 2 of 2 Matching Hosts' and a 'Page Tour' button on the right.

Host	Status	Last Check	Duration	Status Information
localhost	UP	05-26-2026 07:01:23	0d 0h 46m 15s	PING OK - Packet loss = 0%, RTA = 0.07 ms
nagios-client	UP	05-26-2026 06:58:22	0d 0h 9m 55s	NRPE v4.1.0

Nagios CPU load OK

← → ↻ ⚠ Not secure http://13.201.166.116/nagios/ ☆ ⓘ 🔍 ⋮

Nagios®

General
Home
Documentation

Current Status
Tactical Overview
Map (Legacy)
Hosts
Services
Host Groups
Summary
Grid
Service Groups
Summary
Grid
Problems
Services (Unhandled)
Hosts (Unhandled)
Network Outages

Quick Search:

Reports
Availability
Trends (Legacy)
Alerts
History
Summary
Histogram (Legacy)
Notifications
Event Log

System
Comments
Downtime
Process Info
Performance Info
Scheduling Queue
Configuration

Current Network Status
Last Updated: Tue May 26 07:03:41 UTC 2026
Updated every 30 seconds
Nagios® Core™ 4.5.1 - www.nagios.org
Logged in as nagiosadmin
View History For all hosts
View Notifications For All Hosts
View Host Status Detail For All Hosts

Host Status Totals
Up: 2, Down: 0, Unreachable: 0, Pending: 0
All Problems: 0, All Types: 2

Service Status Totals
Ok: 11, Warning: 1, Unknown: 0, Critical: 2, Pending: 0
All Problems: 3, All Types: 14

Service Status Details For All Hosts

Limit Results: 100

Host	Service	Status	Last Check	Duration	Attempt	Status Information
localhost	Current Load	OK	05-26-2026 07:02:38	0d 0h 46m 3s	1/4	OK - load average: 0.00, 0.00, 0.00
	Current Users	OK	05-26-2026 07:03:16	0d 0h 45m 25s	1/4	USERS OK - 2 users currently logged in
	HTTP	WARNING	05-26-2026 07:01:53	0d 0h 41m 48s	4/4	HTTP WARNING: HTTP/1.1 403 Forbidden - 6199 bytes in 0.001 second response time
	PING	OK	05-26-2026 06:59:31	0d 0h 44m 10s	1/4	PING OK - Packet loss = 0%, RTA = 0.05 ms
	Root Partition	OK	05-26-2026 07:00:08	0d 0h 43m 33s	1/4	DISK OK - free space: / 7284 MiB (73.03% inode=99%):
	SSH	OK	05-26-2026 07:00:46	0d 0h 42m 55s	1/4	SSH OK - OpenSSH_9.9 (protocol 2.0)
	Swap Usage	CRITICAL	05-26-2026 06:59:23	0d 0h 39m 18s	4/4	SWAP CRITICAL - 0% free (0 MB out of 0 MB) - Swap is either disabled, not present, or of zero size.
nagios-client	Total Processes	OK	05-26-2026 07:02:01	0d 0h 41m 40s	1/4	PROCS OK: 39 processes with STATE = RSZDT
	CPU Load	OK	05-26-2026 06:58:23	0d 0h 6m 19s+	1/3	OK - load average per CPU: 0.00, 0.00, 0.00
	PING	CRITICAL	05-26-2026 06:54:38	0d 0h 19m 3s	3/3	PING CRITICAL - Packet loss = 100%
	SSH	OK	05-26-2026 06:55:50	0d 0h 17m 51s	1/3	SSH OK - OpenSSH_9.9 (protocol 2.0)
	Total Processes	OK	05-26-2026 06:59:24	0d 0h 6m 19s+	1/3	PROCS OK: 119 processes
	Users	OK	05-26-2026 07:00:25	0d 0h 6m 19s+	1/3	USERS OK - 4 users currently logged in
	Zombie Processes	OK	05-26-2026 07:01:26	0d 0h 6m 19s+	1/3	PROCS OK: 0 processes with STATE = Z

Results 1 - 14 of 14 Matching Services

Page Tour

Nagios CPU load Critical

← → ↻ ⚠ Not secure http://13.201.166.116/nagios/ ☆ ⓘ 🔍 ⋮

Nagios®

General
Home
Documentation

Current Status
Tactical Overview
Map (Legacy)
Hosts
Services
Host Groups
Summary
Grid
Service Groups
Summary
Grid
Problems
Services (Unhandled)
Hosts (Unhandled)
Network Outages

Quick Search:

Reports
Availability
Trends (Legacy)
Alerts
History
Summary
Histogram (Legacy)
Notifications
Event Log

System
Comments
Downtime
Process Info
Performance Info
Scheduling Queue
Configuration

Current Network Status
Last Updated: Tue May 26 07:08:33 UTC 2026
Updated every 30 seconds
Nagios® Core™ 4.5.1 - www.nagios.org
Logged in as nagiosadmin
View History For all hosts
View Notifications For All Hosts
View Host Status Detail For All Hosts

Host Status Totals
Up: 2, Down: 0, Unreachable: 0, Pending: 0
All Problems: 0, All Types: 2

Service Status Totals
Ok: 11, Warning: 1, Unknown: 0, Critical: 2, Pending: 0
All Problems: 3, All Types: 14

Service Status Details For All Hosts

Limit Results: 100

Host	Service	Status	Last Check	Duration	Attempt	Status Information
localhost	Current Load	OK	05-26-2026 07:07:38	0d 0h 50m 55s	1/4	OK - load average: 0.00, 0.00, 0.00
	Current Users	OK	05-26-2026 07:08:16	0d 0h 50m 17s	1/4	USERS OK - 2 users currently logged in
	HTTP	WARNING	05-26-2026 07:06:53	0d 0h 46m 40s	4/4	HTTP WARNING: HTTP/1.1 403 Forbidden - 6199 bytes in 0.001 second response time
	PING	OK	05-26-2026 07:04:31	0d 0h 49m 2s	1/4	PING OK - Packet loss = 0%, RTA = 0.09 ms
	Root Partition	OK	05-26-2026 07:05:08	0d 0h 48m 25s	1/4	DISK OK - free space: / 7284 MiB (73.03% inode=99%):
	SSH	OK	05-26-2026 07:05:46	0d 0h 47m 47s	1/4	SSH OK - OpenSSH_9.9 (protocol 2.0)
	Swap Usage	CRITICAL	05-26-2026 07:04:23	0d 0h 44m 10s	4/4	SWAP CRITICAL - 0% free (0 MB out of 0 MB) - Swap is either disabled, not present, or of zero size.
nagios-client	Total Processes	OK	05-26-2026 07:07:01	0d 0h 45m 32s	1/4	PROCS OK: 39 processes with STATE = RSZDT
	CPU Load	CRITICAL	05-26-2026 07:08:23	0d 0h 0m 10s	1/3	CRITICAL - load average per CPU: 2.04, 1.41, 0.65
	PING	OK	05-26-2026 07:04:38	0d 0h 3m 55s	1/3	PING OK - Packet loss = 0%, RTA = 0.62 ms
	SSH	OK	05-26-2026 07:05:50	0d 0h 22m 43s	1/3	SSH OK - OpenSSH_9.9 (protocol 2.0)
	Total Processes	OK	05-26-2026 06:59:24	0d 0h 11m 11s+	1/3	PROCS OK: 119 processes
	Users	OK	05-26-2026 07:00:25	0d 0h 11m 11s+	1/3	USERS OK - 4 users currently logged in
	Zombie Processes	OK	05-26-2026 07:01:26	0d 0h 11m 11s+	1/3	PROCS OK: 0 processes with STATE = Z

Results 1 - 14 of 14 Matching Services

Page Tour